

CSE_2323

NUMBER SYSTEMS

Digital Logic Design

Course code: CSE-2323

Credit Hour: 3

Md. Mujibur Rahman Maruf

Assistant Lecturer

Dept. of CSE, IIUC

Mujiburmaruf.cuet17@gmail.com

International Islamic University Chittagong(IIUC)

Introduction

- *Name*
- *School*
- *Home town*

Syllabus

Course Content:

#	Content	Duration	CLOs
Mid-Term (30 Marks)			
1	Binary Systems, Boolean Algebra and Logic Gates: Number system, binary codes, binary logics, logic gates, Boolean algebra, canonical and standard forms.	2 weeks	CLO1
2	Simplification of Boolean Functions: The Map Method. Two-, Three-, Four-, Five and Six- variable Maps, Product of Sum Simplification, NAND And NOR Implementation, Don't Care Conditions, Multilevel NAND Circuits, Multilevel NOR Circuits, Exclusive-or and Equivalence Functions.	2 weeks	CLO2
3	Combinational Logic: Design Procedure, Adders, Subtractors, Code Conversion, Analysis Procedure, designing various types of combinational circuit using logic gates.	2 weeks	CLO2

Numerical Representation

2 ways to represent numerical value of quantities

- **Analog**
- **Digital**

*Quantity is represented by
Continuously variable
Ex: mercury thermometer*

*Quantity is represented by
Symbol called digit
Ex. Digital clock
Picking round data from
analog thermometer*

Major difference between analog and digital

Analog=continuous

Digital=descrete(step by step)

Digital and Analog systems

Digital systems

Digital systems operate on discrete elements of information

Numbers (e.g., pocket calculator) -> "digits" -> "digital"

Letters (e.g., word processor)

Pictures (e.g., digital cameras)

- Computers operate on *binary* values (0 and 1)
- Easy to represent binary values electrically
Voltages and currents.

Can be implemented using circuits

Analog system

Analog system operates on quantity that can vary over a continuous range (ex:speaker output)

Advantages and Limitations of Digital systems

Advantages of digital Technique:

- 1. Easier to design
- 2. Information storage is easy
- 3. Accuracy and precision are easier to maintain through the system
- 4. operation can be programmed
- 5. More digital circuit can be fabricated on IC device

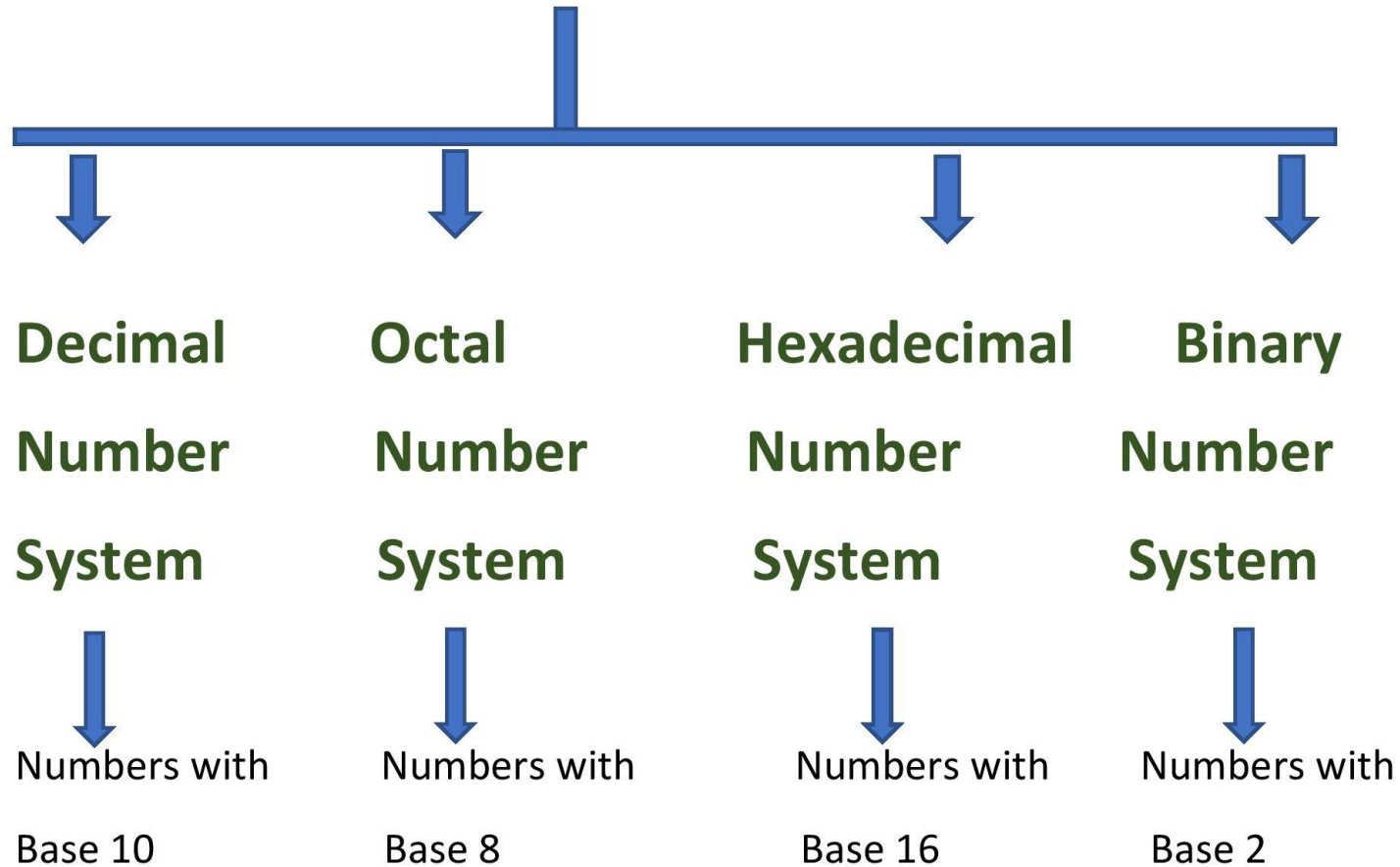
Limitations of digital technique

- 1. the real world is analog
- 2. Processing digitized signals takes time

To take advantages of digital techniques when dealing with analog input and outputs four steps must be followed

- 1. Convert the physical variable to an electrical signal (analog).
- 2. Convert the electrical (analog) signal into digital form. (ADC)
- 3. Process (operate on) the digital information.
- 4. Convert the digital outputs back to real-world analog form. (DAC)

NUMBER SYSTEM



Understanding Decimal Numbers

- Decimal numbers are made of decimal digits: (0,1,2,3,4,5,6,7,8,9)

We are accustomed to using the so-called *decimal number system*.

Ten digits :: 0,1,2,3,4,5,6,7,8,9

Every digit position has a weight which is a power of 10.

Base or radix is 10.

- But how many items does a decimal number represent?

$$8653 = 8 \times 10^3 + 6 \times 10^2 + 5 \times 10^1 + 3 \times 10^0$$

- What about fractions?

$$97654.35 = 9 \times 10^4 + 7 \times 10^3 + 6 \times 10^2 + 5 \times 10^1 + 4 \times 10^0 + 3 \times 10^{-1} + 5 \times 10^{-2}$$

In formal notation -> $(97654.35)_{10}$

- Why do we use 10 digits, anyway?

Understanding Octal Numbers

- Octal numbers are made of octal digits: (0,1,2,3,4,5,6,7)
- How many items does an octal number represent?
 $(4536)_8 = 4 \times 8^3 + 5 \times 8^2 + 3 \times 8^1 + 6 \times 8^0 = (1362)_{10}$
- What about fractions?
 $(465.27)_8 = 4 \times 8^2 + 6 \times 8^1 + 5 \times 8^0 + 2 \times 8^{-1} + 7 \times 8^{-2}$

- Octal numbers don't use
- digits 8 or 9

<u>Decimal</u>	<u>Octal</u>
0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	10
9	11
10	12
11	13
12	14
13	15
14	16
15	17

Understanding Hexa Decimal Numbers

Name

- "hexa" (Greek) => six
- "decem" (Latin) => ten

Characteristics

- Sixteen symbols
 - 0 1 2 3 4 5 6 7 8 9 A B C D E F

<u>Decimal</u>	<u>Hex</u>	<u>Decimal</u>	<u>Hex</u>
0	0	16	10
1	1	17	11
2	2	18	12
3	3	19	13
4	4	20	14
5	5	21	15
6	6	22	16
7	7	23	17
8	8	24	18
9	9	25	19
10	A	26	1A
11	B	27	1B
12	C	28	1C
13	D	29	1D
14	E	30	1E
15	F	31	1F

Binary Number System

Counting with Binary Numbers

- Binary numbers are made of binary digits (bits):
0 and 1

Every digit position has a weight which is a power of 2.
Base or radix is 2.

- How many items does an binary number represent?

$$(1011)_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = (11)_{10}$$

- What about fractions?

$$(110.10)_2 = 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2}$$

- Groups of eight bits are called a **byte**

$$(11001001)_2$$

- Groups of four bits are called a **nibble**.

$$(1101)_2$$

<u>Decimal</u>	<u>Binary</u>
0	0
1	1
2	10
3	11
4	100
5	101
6	110
7	111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111

Why Use Binary Numbers?

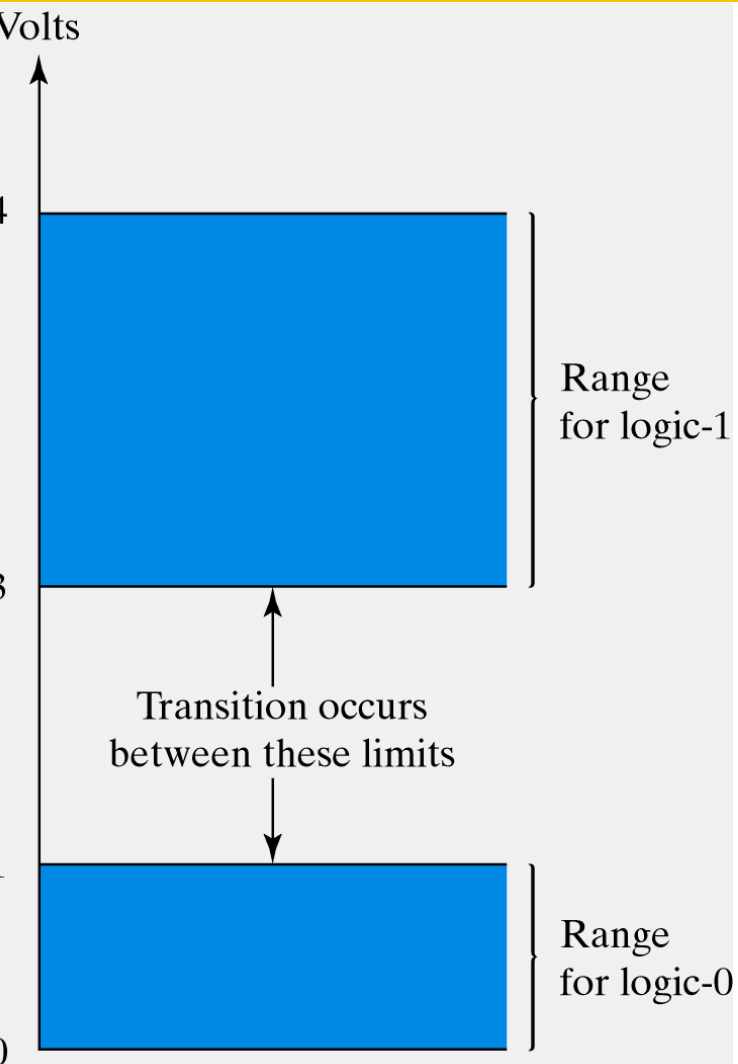


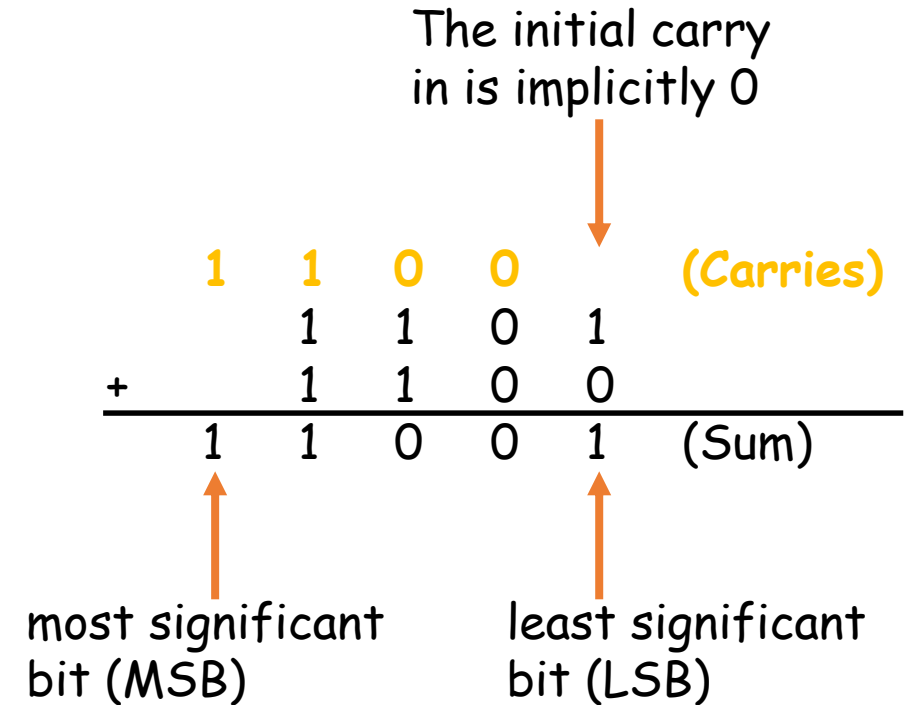
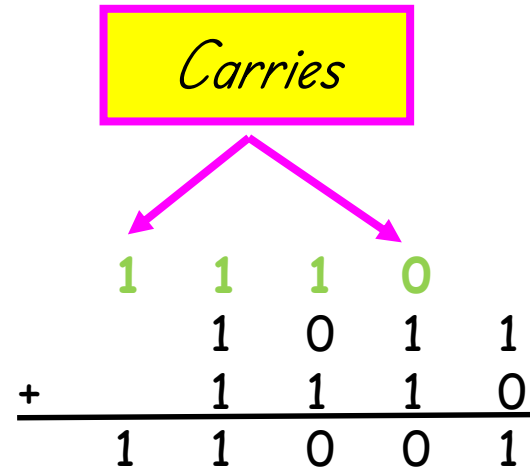
Fig. 1-3 Example of binary signals

- *Easy to represent 0 and 1 using electrical values.*
- *Possible to tolerate noise.*
- *Easy to transmit data*
- *Easy to build binary circuits.*

Binary addition: Another example

$0 + 0 = 0$
 $0 + 1 = 1$
 $1 + 0 = 1$
 $1 + 1 = 10$

carry



Multiplication and Division with base

- Multiplication with 10 (decimal system)

$$435 \times 10 = 4350$$

*Left Shift and add
zero at right end*

- Multiplication with 10 (=2) (binary system)

$$1101 \times 10 = 11010$$

- Division by 10 (decimal system)

$$435 / 10 = 43.5$$

*Right shift and drop
right most digit or
shift after decimal
point*

- Division by 10 (=2) (binary system)

$$1101 / 10 = 110.1$$

Binary-to-Decimal Conversion

Each digit position of a binary number has a weight.

Some power of 2.

A binary number:

$$B = b_{n-1} b_{n-2} \dots b_1 b_0 . b_{-1} b_{-2} \dots b_{-m}$$

Corresponding value in decimal:

$$D = \sum b_i 2^i$$

1. $101011 \rightarrow 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 43$

$$(101011)_2 = (43)_{10}$$

2. $.0101 \rightarrow 0 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4} = .3125$

$$(.0101)_2 = (.3125)_{10}$$

3. $101.11 \rightarrow 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} = 5.75$

$$(101.11)_2 = (5.75)_{10}$$

Examples

Decimal-to-Binary Conversion

Consider the integer and fractional parts separately.

For the integer part,

Repeatedly divide the given number by 2, and go on accumulating the remainders, until the number becomes zero.

Arrange the remainders *in reverse order*.

For the fractional part,

Repeatedly multiply the given fraction by 2.

Accumulate the integer part (0 or 1).

If the integer part is 1, chop it off.

Arrange the integer parts *in the order* they are obtained.

Decimal-to-Binary Conversion

Example 1

2	239	
2	119	--- 1
2	59	--- 1
2	29	--- 1
2	14	--- 1
2	7	--- 0
2	3	--- 1
2	1	--- 1
2	0	--- 1



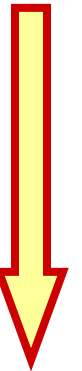
$$(239)_{10} = (11101111)_2$$

2	64	
2	32	--- 0
2	16	--- 0
2	8	--- 0
2	4	--- 0
2	2	--- 0
2	1	--- 0
2	0	--- 1



$$(64)_{10} = (1000000)_2$$

.634	x 2	=	1.268
.268	x 2	=	0.536
.536	x 2	=	1.072
.072	x 2	=	0.144
.144	x 2	=	0.288
:			
:			



$$(.634)_{10} = (.10100.....)_2$$

Decimal-to-Hex Conversion

$$(2297)_{10} = (?)_{16}$$

1 Successive division

2 Note remainders. Change to hexadecimal.

$$\begin{array}{r} 16 \overline{) 2297} \\ 16 \overline{) 143} \\ 16 \overline{) 8} \end{array}$$

$$\begin{array}{l} 9 \rightarrow 9 \\ 15 \rightarrow F \\ 8 \rightarrow 8 \end{array}$$

4 Read bottom up

$$= 8F9$$

3 Stop dividing
 $\therefore Q = 0$

$$\therefore (2297)_{10} = (8F9)_{16}$$

Decimal to Hexadecimal

$$\begin{array}{r} 16 \overline{) 650} \\ 16 \overline{) 40} \text{ --- } 10=A \\ 16 \overline{) 2} \text{ --- } 8 \\ 0 \text{ --- } 2 \end{array}$$

$$(650)_{10} = (28A)_{16}$$

$$\begin{array}{r} 16 \overline{) 910} \\ 16 \overline{) 56} \text{ --- } (14)E \\ 16 \overline{) 3} \text{ --- } 8 \\ 2 \overline{) 0} \text{ --- } 3 \end{array}$$

$$(910)_{10} = (38E)_{16}$$

$$.634 \times 16 = 10.144(A)$$

$$.144 \times 16 = 2.304$$

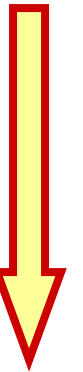
$$.304 \times 16 = 4.864$$

$$.864 \times 16 = 13.824(D)$$

:

:

$$(.634)_{10} = (.A24D.....)_{16}$$



Decimal-to-Octal Conversion

$$\begin{array}{r|l} 8 & 540 \\ 8 & 67 \text{ --- } 4 \\ 8 & 8 \text{ --- } 3 \\ 8 & 1 \text{ --- } 0 \\ & 0 \text{ --- } 1 \end{array}$$

$(540)_{10} = (1034)_8$



<u>Decimal</u>	<u>Binary</u>
0	0
1	1
2	10
3	11
4	100
5	101
6	110
7	111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111

$$.30 \times 8 = 2.4$$

$$.4 \times 8 = 3.2$$

$$.2 \times 8 = 1.6$$

$$.6 \times 8 = 4.6$$

:

:

$$(.634)_{10} = (.2314.....)_8$$



Conversion Between Number Bases



$$(6603)_8 = (\underline{110} \ \underline{110} \ \underline{000} \ \underline{011})_2 = (110110000011)_2 = \underline{1101} \ \underline{1000} \ \underline{0011} = (D83)_{16}$$

$$(621)_8 = (\underline{110} \ \underline{010} \ \underline{001})_2 = (110010001)_2 = \underline{0001} \ \underline{1001} \ \underline{0001} = (191)_{16}$$

$$(.41)_8 = (. \underline{100} \ \underline{001})_2 = (.100001)_2 = (. \underline{1000} \ \underline{0100})_2 = (.84)_{16}$$

$$(5.274)_8 = (\underline{101} . \underline{010} \ \underline{111} \ \underline{100})_2 = (101.0101111)_2 = (\underline{0101} . \underline{0101} \ \underline{1110})_2 = (5.5E)_{16}$$

Ways to Signed Number representation

Since Binary number can have only two symbols either 0 or 1 for each position or bit, so it is not possible to add minus or plus symbols in front of a binary number

Negative and positive numbers can be distinguishable with the help of an extra bit or flag called sign bit or sign flag in the Binary number representation system for signed numbers. The sign bit is 0 for positive numbers and 1 for negative binary numbers

Ways to represent Signed Number

- Signed Magnitude Method
- 1's Complement Method
- 2's Complement Method

Sign-magnitude Method

We only add an extra sign bit to recognize negative and positive numbers. Sign bit has 1 for negative number and 0 for positive number

For an n -bit number representation

The most significant bit (MSB) indicates sign

0 $\rightarrow$ positive

1 $\rightarrow$ negative

The remaining $n-1$ bits represent magnitude.

Range of numbers that can be represented:

Maximum $:: + (2^{n-1} - 1)$

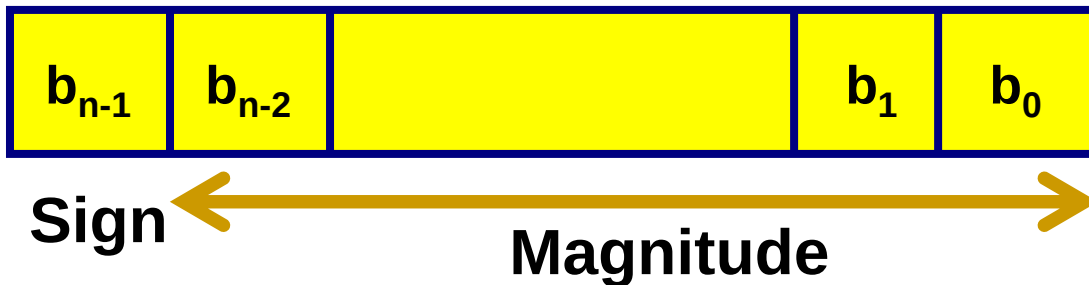
Minimum $:: - (2^{n-1} - 1)$

A problem:

Two different representations of zero.

+0 $\rightarrow$ 0 000...0

-0 $\rightarrow$ 1 000...0



One's Complement Representation

Basic idea:

Positive numbers are represented exactly as in sign-magnitude form.

Negative numbers are represented in 1's complement form.

How to compute the 1's complement of a number?

Complement every bit of the number ($1 \rightarrow 0$ and $0 \rightarrow 1$).

MSB will indicate the sign of the number.

0 $\rightarrow$ positive

1 $\rightarrow$ negative

One's Complement Representation

Range of numbers that can be represented:

Maximum :: $+(2^{n-1} - 1)$

Minimum :: $-(2^{n-1} - 1)$

A problem:

Two different representations of zero.

+0 $\rightarrow$ 0 000....0

-0 $\rightarrow$ 1 111....1

Advantage of 1's complement representation

Subtraction can be done using addition.

Leads to substantial saving in circuitry

0000 $\rightarrow$ +0

0001 $\rightarrow$ +1

0010 $\rightarrow$ +2

0011 $\rightarrow$ +3

0100 $\rightarrow$ +4

0101 $\rightarrow$ +5

0110 $\rightarrow$ +6

0111 $\rightarrow$ +7

1000 $\rightarrow$ -7

1001 $\rightarrow$ -6

1010 $\rightarrow$ -5

1011 $\rightarrow$ -4

1100 $\rightarrow$ -3

1101 $\rightarrow$ -2

1110 $\rightarrow$ -1

1111 $\rightarrow$ -0

To find the representation of -4, first note that

+4 = 0100

-4 = 1's complement of 0100 = 1011

Subtraction Using Addition :: 1's Complement

How to compute $A - B$?

Compute the 1's complement of B (say, B_1).

Compute $R = A + B_1$

If the carry obtained after addition is '1'

Add the carry back to R (called *end-around carry*).

That is, $R = R + 1$.

The result is a positive number.

Else

The result is negative, and is in 1's complement form.

$A = 6$ (0110)

$B = 2$ (0010)

$6 - 2 = A - B$

1's complement of 2 = 1101

A 6 :: 0110

B_1 -2 :: 1101

1 0011

R 1

0100 → +4

Assume 4-bit representations.

Since there is a carry, it is added back to the result.

The result is positive.

Two's Complement Representation

Basic idea:

Positive numbers are represented exactly as in sign-magnitude form.

Negative numbers are represented in 2's complement form.

How to compute the 2's complement of a number?

Complement every bit of the number ($1 \rightarrow 0$ and $0 \rightarrow 1$), and then *add one* to the resulting number.

MSB will indicate the sign of the number.

0 $\rightarrow$ positive

1 $\rightarrow$ negative

0000 $\rightarrow$ +0 1000 $\rightarrow$ -8

0001 $\rightarrow$ +1 1001 $\rightarrow$ -7

0010 $\rightarrow$ +2 1010 $\rightarrow$ -6

0011 $\rightarrow$ +3 1011 $\rightarrow$ -5

0100 $\rightarrow$ +4 1100 $\rightarrow$ -4

0101 $\rightarrow$ +5 1101 $\rightarrow$ -3

0110 $\rightarrow$ +6 1110 $\rightarrow$ -2

0111 $\rightarrow$ +7 1111 $\rightarrow$ -1

To find the representation of, say, -4, first note that

$$+4 = 0100$$

$$-4 = 2's \text{ complement of } 0100 = 1011 + 1 = 1100$$

Subtraction Using Addition :: 2's Complement

How to compute $A - B$?

Compute the 2's complement of B (say, B_2).

Compute $R = A + B_2$

Ignore carry if it is there.

The result is in 2's complement form.

$$A = 6 \text{ (0110)}$$

$$B = 2 \text{ (0010)}$$

$$6 - 2 = A - B$$

$$1's \text{ complement of } 2 = 1101$$

$$2's \text{ complement of } 2 = 1101 + 1 = 1110$$

$$\begin{array}{rcl} 6 & :: & 0110 & A \\ -2 & :: & 1110 & B_2 \\ \hline & & 10100 & R \end{array}$$

Ignore carry

+4

Comparison between 1's and 2's Complement

1's complement is easier to implement by digital computer than 2's complement

The 1's complement has the advantage of being easier to implement by digital components since the only thing that must be done is to change 0's into 1's and 1's into 0's.

The implementation of 2's complement may be obtained by adding 1 to the least significant digit of the 1's complement and

Subtraction by 2's complement requires less arithmetic operation.

During subtraction of two numbers by complements, the 2's complement is advantageous in that only one arithmetic operation is required. The 1's complement requires two arithmetic additions when an end-around carry occurs.

Comparison between 1's and 2's Complement

1's complement has two arithmetic zeros.

The 1's complement has the additional disadvantages of possessing two arithmetic zeros: one with all 0's and one with all 1's.

While the 2's complement has only one arithmetic zero, the 1's complement zero can be positive or negative, which may complicate matters.

BINARY CODES

Digital Codes or Binary Codes:

A group of binary digits is known as a binary code or a digital code.

Numeric Code:

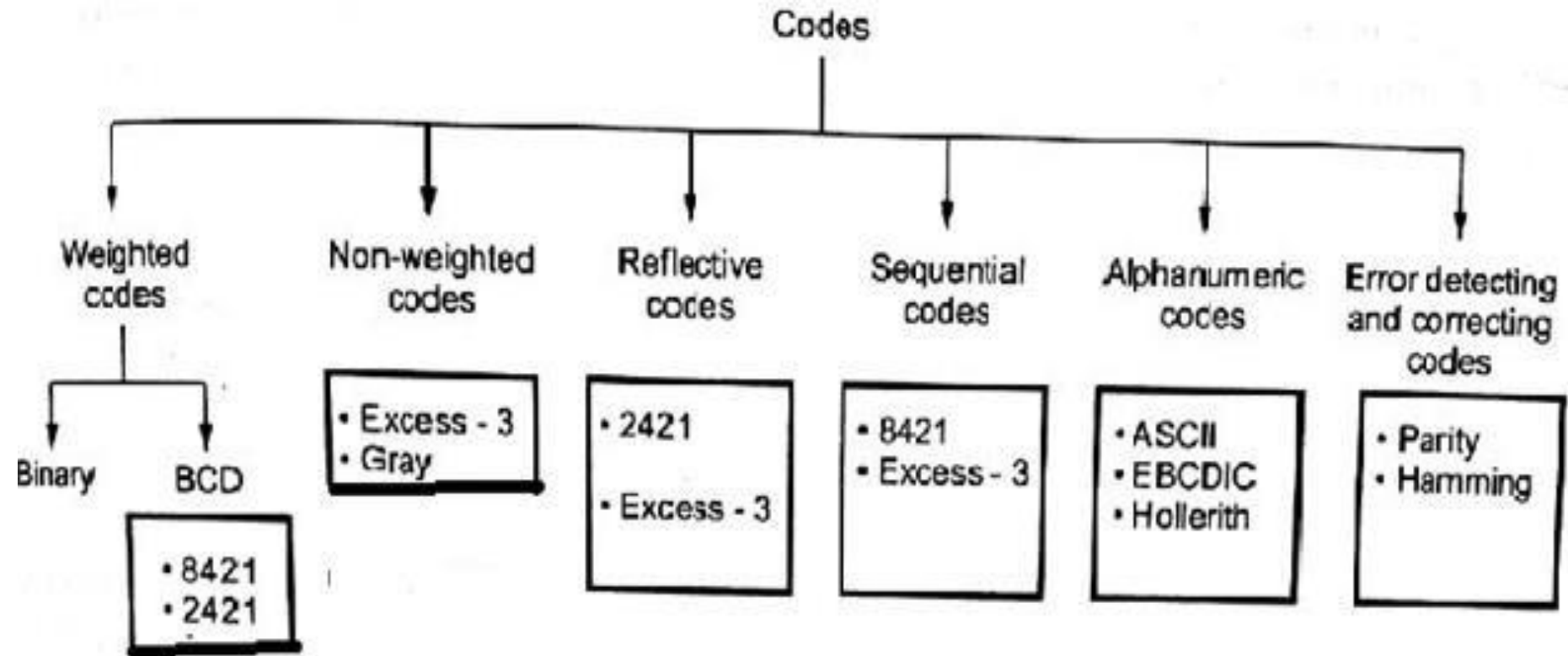
Digital code represented by the number is known as a numeric code.

Classification:

The codes are broadly classified into

1. Weighted code
2. Non-weighted code
3. Reflective code
4. Sequential code
5. Error detecting and correcting codes

Classification



Weighted codes

(Examples: 8421, 2421 Code)

the position of each number represent a specific weight.

In these codes each decimal digit is represented by a group of four bits. **called Nibble**

each digit is assigned a specific weight according to its position

For example, in 8421/BCD code,

1001 the weights of 1, 1, 0, 1 (from left to right) are 8, 4, 2 and 1 respectively.

BCD and 2421 Code								
Decimal Equivalent	BCD Code				2421 Code			
	D_4	D_3	D_2	D_1	Y_4	Y_3	Y_2	Y_1
0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
2	0	0	1	0	0	0	1	0
3	0	0	1	1	0	0	1	1
4	0	1	0	0	0	1	0	0
5	0	1	0	1	1	0	1	1
6	0	1	1	0	1	1	0	0
7	0	1	1	1	1	1	0	1
8	1	0	0	0	1	1	1	0
9	1	0	0	1	1	1	1	1

8421 or BCD code

BCD stands for Binary-Coded Decimal.

A BCD number is a four-bit binary group that represents one of the ten decimal digits 0 through 9.

Decimal Equivalent	BCD Code			
	D_4	D_3	D_2	D_1
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

Example:

Decimal number 4926

4

9

2

6



8421 BCD coded number

0100 1001 0010 0110

8421 or BCD code

- (BCD) is a type of binary code used to represent a given decimal number in an equivalent binary form.
- The BCD equivalent of a decimal number is written by replacing each decimal digit in the integer and fractional parts with its four-bit binary equivalent.
- The BCD code described above is more precisely known as the 8421 BCD code .

As an example, the BCD equivalent of (23.15)₁₀ is written as (0010 0011.0001 0101)_{BCD}

BCD Addition

Addition of BCD (8421) is performed by adding two digits of binary, starting from least significant digit. In case if the result is an **illegal code** (**greater than 9**) or if there is a carry out of one then **add 0110(6)** and add the resulting carry to the next most significant.

For example:-

6	0	1	1	0	← BCD for 6
+ 3	0	0	1	1	← BCD for 3
9	1	0	0	1	← BCD for 9

6	0	1	1	0	← BCD for 6
+ 8	1	0	0	0	← BCD for 8
14	1	1	1	0	← Invalid BCD number

6	0	1	1	0	← BCD for 6
+ 8	1	0	0	0	← BCD for 8
14	1	1	1	0	← Invalid BCD number
	0	1	1	0	← Add 6 for correction
0	0	0	1	0	← BCD for 14
1					4

BCD Addition

For example:-

Add 8765 with 3943 using BCD addition.

Solution:-

	$\begin{array}{r} 111 \\ 1000 \end{array}$	$\begin{array}{r} 111 \\ 0111 \end{array}$	$\begin{array}{r} 1 \\ 0110 \end{array}$	$\begin{array}{r} 111 \\ 0101 \end{array}$
	$+ 0011$	$+ 1001$	$+ 0100$	$+ 0011$
	$\begin{array}{r} 1 \\ 1100 \end{array}$	$\begin{array}{r} 1 \\ 0001 \end{array}$	$\begin{array}{r} 1 \\ 1010 \end{array}$	$\begin{array}{r} 1 \\ 1000 \end{array}$
Correction $\rightarrow$	0110	0110	0110	0000
	$\begin{array}{r} 0001 \\ 1 \end{array}$	$\begin{array}{r} 0010 \\ 2 \end{array}$	$\begin{array}{r} 0111 \\ 7 \end{array}$	$\begin{array}{r} 0000 \\ 0 \end{array}$
				$\begin{array}{r} 1000 \\ 8 \end{array}$